

1] Multiplication of two matrices and transpose of a Matrix

```
#include<stdio.h>

#define max 5

void read_matrix(int a[max][max],int m,int n);

void write_matrix(int a[max][max],int m,int n);

void tran_matrix(int a[max][max],int m,int n);

void multi_matrix(int a[max][max],int b[max][max],int x,int y,int z);

int main()

{

int a[max][max],b[max][max],m,n,x,y,choice;

printf("Enter order of matrix A (m,n)");

scanf("%d%d",&m,&n);

printf("Enter order of matrix B (p,q)");

scanf("%d%d",&x,&y);

read_matrix(a,m,n);

read_matrix(b,x,y);

printf("Enter 1 for Transpose \nEnter 2 for Multiplication ");

scanf("%d",&choice);

switch(choice)

{

case 1:

printf("Transpose of matrix a\n");

tran_matrix(a,m,n);

printf("Transpose of matrix b\n");

tran_matrix(b,x,y);

break;

case 2:

if(n==x)
```

```

multi_matrix(a,b,m,y,n);
else
{
printf("Dimension mismatch");
printf("Enter order such that n==x");
}
break;
default:
printf("Invalid choice");
}
return 0;
}

void read_matrix(int a[max][max],int m,int n)
{
int i,j;
for(i=0;i<m;i++)
{
printf("Enter row %d elements of matrix A",i+1);

for(j=0;j<n;j++)
scanf("%d",&a[i][j]);
}
}

void write_matrix(int a[max][max],int m,int n)
{
int i,j;
for(i=0;i<m;i++)
{

```

```

for(j=0;j<n;j++)
printf("%d\t",a[i][j]);
printf("\n");
}
}

void tran_matrix(int a[max][max],int m,int n)
{
int i,j,c[max][max];
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
c[j][i]=a[i][j];
}
write_matrix(c,n,m);
}

void multi_matrix(int a[max][max],int b[max][max],int x,int y,int z)
{
int i,j,k,c[max][max];
for(i=0;i<x;i++)
for(j=0;j<y;j++)
{
c[i][j]=0;
for(k=0;k<z;k++)
c[i][j]=c[i][j]+a[i][k]*b[k][j];
}
write_matrix(c,x,y);
}

```

2] calculator implementation in C using pointers

```
#include<stdio.h>
```

```
float add(int a,int b);
```

```
float sub(int a,int b);
```

```
float mult(int a,int b);
```

```
float div(int a,int b);
```

```
void display(float display);
```

```
int main()
```

```
{
```

```
    float(*p)(int,int);
```

```
    void (*s)(float);
```

```
    char ch;
```

```
    float res;
```

```
    int n1,n2;
```

```
    printf("Enter two numbers");
```

```
    scanf("%d %d",&n1,&n2);
```

```
    getchar();
```

```
    printf("Enter \n+\n-\n*\n/\n");
```

```
    scanf("%c",&ch);
```

```
    switch(ch)
```

```
    {
```

```
        case'+':
```

```
        p=add;
```

```
        break;
```

```
        case'-':
```

```
        p=sub;
```

```
        break;
```

```
    case'*':
        p=mult;
        break;
    case'/':
        p=div;
        break;
    }
    res=(*p)(n1,n2);
    s=&display;
    (*s)(res);
    return 0;
}

float add(int a,int b)
{
    return a+b;
}

float sub(int a,int b)
{
    return a-b;
}

float mult(int a,int b)
{
    return a*b;
}

float div(int a,int b)
{
    if(b==0)
        return 0;
```

```
    else
        return (float)a/b;
}
void display(float res)
{
    printf("Result=%f",res);
}
```

3] Simple banking application in C by making use of array of Structures

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
void creation();
void deposit();
void withdraw();
void balance();
int a=0,i=101,tot=0;
struct bank
{
    int no;
    char name[30];
    float bal;
} s[20];
int main()
{
    int ch;
    while (1)
    {
```

```
printf("\n*****\n");
printf("\nBANKING");
printf("\n1.Creation");
printf("\n2.Deposit");
printf("\n3.Withdraw");
printf("\n4.Enquiry");
printf("\n5.Exit");
printf("\nEnter your choice: ");
scanf("%d",&ch);
printf("\n*****");
switch(ch)
{
    case 1: creation(); break;
    case 2: deposit(); break;
    case 3: withdraw(); break;
    case 4: balance(); break;
    case 5: exit(0);
    default: printf("\nEnter 1-5\n");
}
}
}

void creation()
{
    printf("\n*****");
    printf("\nACCOUNT CREATION:");
    printf("\nYOUR ACCOUNT NO.=%d",i);
    s[a].no = i;
```

```

printf("\nEnter your name: ");
scanf("%s", s[a].name);
printf("\nYour minimum deposit is RS 500");
s[a].bal = 500;
printf("\nA/c Created successfully");
printf("\n_____");
tot = a;
a++;
i++;
}
void deposit()
{
    int an,m;
    float am;
    printf("\n*****\nDEPOSIT");
    printf("\nEnter your A/c no: ");
    scanf("%d",&an);
    for(m=0;m<=tot;m++)
    {
        if(s[m].no==an)
        {
            printf("\nA/c NO.:%d",s[m].no);
            printf("\nName:%s",s[m].name);
            printf("\nA/c Balance=%.2f",s[m].bal);
            printf("\nEnter amount to deposit: ");
            scanf("%f",&am);
            s[m].bal += am;

```



```

        printf("\nCurrent balance=%.2f\n_____",s[m].bal);
        return;
    }
}

printf("\nINVALID A/c No.");
}

void withdraw()
{
    int an,m;
    float am;

    printf("\n*****\nWithdraw");
    printf("\nEnter your A/c no: ");
    scanf("%d",&an);
    for(m=0;m<=tot;m++)
    {
        if(s[m].no==an)
        {
            printf("\nA/c NO.:%d",s[m].no);
            printf("\nName:%s",s[m].name);
            printf("\nA/c Balance=%.2f",s[m].bal);
            printf("\nEnter amount to withdraw: ");
            scanf("%f",&am);
            if ((s[m].bal - am) < 500)
            {
                printf("\nINSUFFICIENT BALANCE");
                return;
            }
        }
    }
}

```

```

        s[m].bal -= am;
        printf("\nWithdraw successful");
        printf("\nBalance=%.2f\n_____",s[m].bal);
        return;
    }
}
printf("\nInvalid Account no");
}
void balance()
{
    int an,m;
    printf("\nEnter account number: ");
    scanf("%d",&an);
    for(m=0;m<=tot;m++)
    {
        if(s[m].no==an)
        {
            printf("\nA/c NO:%d",s[m].no);
            printf("\nName:%s",s[m].name);
            printf("\nBalance=%.2f",s[m].bal);
            return;
        }
    }
    printf("\nInvalid Account no");
}

```

4] Program to implement a stack using an array.

/* Stack using an ARRAY */

```
#include <stdio.h>

#define MAX 5

int top = -1;

int stack_arr[MAX];

/* Begin of push */

void push()

{

    int pushed_item;

    if (top == (MAX - 1))

        printf("Stack Overflow\n");

    else

    {

        printf("Enter the item to be pushed in stack: ");

        scanf("%d", &pushed_item);

        top = top + 1;

        stack_arr[top] = pushed_item;

    }

}

/* End of push */

/* Begin of pop */

void pop()

{

    if (top == -1)

        printf("Stack Underflow\n");

    else

    {

        printf("Popped element is : %d\n", stack_arr[top]);

        top = top - 1;

    }

}
```

```
    }  
}  
/* End of pop */  
/* Begin of display */  
void display()  
{  
    int i;  
    if (top == -1)  
        printf("Stack is empty\n");  
    else  
    {  
        printf("Stack elements :\n");  
        for (i = top; i >= 0; i--)  
            printf("%d\n", stack_arr[i]);  
    }  
}  
/* End of display */  
/* Begin of main */  
int main()  
{  
    int choice;  
    do  
    {  
        printf("1. Push\n");  
        printf("2. Pop\n");  
        printf("3. Display\n");  
        printf("4. Quit\n");  
        printf("Enter your choice : ");
```

```
scanf("%d", &choice);
switch (choice)
{
    case 1:
        push();
        break;
    case 2:
        pop();
        break;
    case 3:
        display();
        break;
    case 4:
        break;
    default:
        printf("Wrong choice\n");
}
} while (choice != 4);
return 0;
}

/* End of main */
```

5] Program to implement a Queue using an array.

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#define max 5
```

```
static int queue[max];
```

```
int front = -1;
```

```
int rear = -1;

void insert(int x);

int dequeue();

void view();

int main() {
    int ch = 0, val;
    while (ch != 4) {
        printf("\nQUEUE OPERATION\n");
        printf("\n1.Insert\n2.Delete\n3.View\n4.Quit\n");
        printf("\nEnter choice:\n");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
                printf("\nEnter element to be inserted\n");
                scanf("%d", &val);
                insert(val);
                break;
            case 2:
                val = dequeue();
                if (val != -1)
                    printf("\nElement deleted: %d\n", val);
                break;
            case 3:
                view();
                break;
            case 4:
                exit(0);
        }
    }
}
```

```
        default:
            printf("\nInvalid choice\n");
        }
    }
    return 0;
}

void insert(int x) {
    if (rear == max - 1) {
        printf("\nQueue is Full\n");
        return;
    }
    queue[++rear] = x;
    if (front == -1)
        front = 0;
}

int dequeue() {
    int val;
    if (front == -1) {
        printf("\nQueue Empty\n");
        return -1;
    }
    val = queue[front];
    if (front == rear)
        front = rear = -1;
    else
        front++;
    return val;
}
```

```

void view() {
    int i;
    if (front == -1) {
        printf("\nQueue Empty\n");
    } else {
        printf("\nFront--> ");
        for (i = front; i <= rear; i++) {
            printf("%d ", queue[i]);
        }
        printf("<--Rear\n");
    }
}

```

6] Program to implement array of objects to process CIE Data.

```

#include<iostream>
#include<cstring>
using namespace std;
class STUDENT
{
private:
    int rno;
    char stud_name[10];
    char usn[10];
    int IA1;
    int IA2;
    int OBA1;
    int OBA2;

```



```
int CS;

int total;

public:

void read_data()
{
    cout<<"Enter Student's Roll Number:";

    cin>>rno;

    cout<<"Enter Student Name:";

    cin>>stud_name;

    cout<<"Enter Student USN:";

    cin>>usn;

    cout<<"Enter IA1 Marks(30):";

    cin>>IA1;

    cout<<"Enter IA2 Marks(30):";

    cin>>IA2;

    cout<<"Enter OBA1 Marks(10):";

    cin>>OBA1;

    cout<<"Enter OBA2 Marks(10):";

    cin>>OBA2;

    cout<<"Enter Course Project/Seminar Marks(20):";

    cin>>CS;

    total=IA1+IA2+OBA1+OBA2+CS;
}

void write_data()
{
    cout<<rno<<"\t";

    cout<<stud_name;
```

```

        if(strlen(stud_name) < 8)
            cout<<"\t\t";
        else
            cout<<"\t";
        cout<<usn;
        if(strlen(usn) < 12)
            cout<<"\t\t";
        else
            cout<<"\t";
        cout<<IA1<<"\t";
        cout<<IA2<<"\t";
        cout<<OBA1<<"\t";
        cout<<OBA2<<"\t";
        cout<<CS<<"\t";
        cout<<total;
    }
};

int main()
{
    STUDENT S[10];
    int n;
    cout<<"Enter Number of Students:\n";
    cin>>n;
    for (int i = 0; i < n; i++)
    {
        S[i].read_data();
    }
}

```



```
float area(int base, float h)
{
    float a;
    a=(0.5*base*h);
    return(a);
}
int main()
{
    float a;
    int choice,l, b, base;
    float r, h;
    cout << "\n1.Rectangle\n2.Circle\n3.Triangle\n";
    cout << "\nEnter your choice: ";
    cin >> choice;
    switch(choice)
    {
        case 1:
            cout << "\nEnter the values for\nlength:";
            cin >> l;
            cout << "breadth:";
            cin >> b;
            a = area(l, b);
            cout << "\nArea of rectangle:" << a;
            break;
        case 2:
            cout<<"Enter the Radius\n";
            cin>>r;
            a=area(r);
```

```

        cout<<"\nArea of Circle:"<<a;
        break;
    case 3:
        cout<<"\nEnter base and Height of a Triangle\n";
        cin>>base>>h;
        a=area(base,h);
        cout<<"\nArea of Triangle:"<<a;
        break;
    default:
        cout<<"Choice is Invalid!!"<<endl;
    }
    return 0;
}

```

8] Program to implement the concept of passing objects to function by 1. Value 2. reference. Write a C++ program to illustrate the passing of objects as arguments. The user should input two different times in hours and minutes. The program should find the sum of the two times and display the result in hours and minutes.

```

#include <iostream>
using namespace std;
class Time
{
private:
    int hours, mins;
public:
    void get_time(int h, int m);

```

```
void put_time();  
void add_time(Time t1, Time t2);  
}  
void Time::get_time(int h, int m)  
{  
    hours = h;  
    mins = m;  
}  
void Time::put_time()  
{  
    cout << hours << " hours and " << mins << " minutes" << "\n";  
}  
void Time::add_time(Time t1, Time t2)  
{  
    int minutes;  
    minutes = t1.mins + t2.mins;  
    hours = minutes / 60;  
    mins = minutes % 60;  
    hours = hours + t1.hours + t2.hours;  
}  
int main()  
{  
    Time T1, T2, T3;  
    T1.get_time(5, 45);  
    T2.get_time(1, 30);  
    T3.add_time(T1, T2);  
    T1.put_time();  
    T2.put_time();
```

```
T3.put_time();  
return 0;  
}
```

9] Program to implement the hybrid inheritance Write a C++ program for hybrid inheritance as shown

```
#include<iostream>  
  
using namespace std;  
  
class student  
{  
    protected :  
        char ch[10];  
        int Roll_number;  
    public:  
        void get_data();  
        void put_data();  
};  
  
void student::get_data()  
{  
    cout<<"Enter the name and roll number of student:";  
    cin>>ch>>Roll_number;  
}  
  
void student::put_data()  
{  
    cout<<"Name="<<ch<<"\n";  
    cout<<"Roll_number="<<Roll_number<<"\n";  
}  
  
class electives:virtual public student
```

```

{
    protected:
        int elect1,elect2;
    public:
        void get_emarks();
        void put_emarks();
};

void electives::get_emarks()
{
    cout<<"Enter values of elective 1 and elective 2:";
    cin>>elect1>>elect2;
}

void electives::put_emarks()
{
    cout<<"Marks in elective1="<<elect1<<"\n";
    cout<<"Marks in elective2="<<elect2<<"\n";
}

class core:virtual public student
{
    protected:
        int core1,core2,core3;
    public:
        void get_cmarks();
        void put_cmarks();
};

void core::get_cmarks()
{
    cout<<"Enter values of core 1,core2 and core3:";

```



```

        cin>>core1>>core2>>core3;
    }
    void core::put_cmarks()
    {
        cout<<"Marks in core1="<<core1<<"\n";
        cout<<"Marks in core2="<<core2<<"\n";
        cout<<"Marks in core3="<<core3<<"\n";
    }
    class result:public electives,public core
    {
        public:
            void display();
    };
    void result::display()
    {
        if (elect1>=40 && elect2>=40 && core1>=40 &&core2>=40 && core3>=40)
            cout<<"Result is pass\n";
        else
            cout<<"Result is fail\n";
    }
    int main()
    {
        result Atharv;
        Atharv.get_data();
        Atharv.get_emarks();
        Atharv.put_emarks();
        Atharv.get_cmarks();
        Atharv.put_cmarks();
    }

```

```
        Atharv.display();  
        return 0;  
    }
```

10] Program to illustrate the use of operator overloading.

```
#include<iostream>  
  
using namespace std;  
  
class complex  
{  
    float x;  
    float y;  
public:  
    complex(){} // default constructor  
    complex (float real, float imag) // parameterized constructor  
    {  
        x=real;  
        y=imag;  
    }  
    complex operator + (complex);  
    void display(void);  
};  
  
complex complex :: operator + (complex c)  
{  
    complex temp;  
    temp.x = x + c.x;  
    temp.y = y + c.y;  
    return(temp);  
}
```

```
}  
  
void complex :: display(void)  
{  
    if(y>0)  
        cout<<x<<"+"<<y<<"\n";  
    else  
        cout<<x<<"-"<<-1*y<<"\n";  
}  
  
int main()  
{  
    complex C1,C2,C3;  
    C1=complex(2.5, 3.5);  
    C2=complex(1.6, 2.7);  
    C3=C1+C2;  
    cout<<"C1=";  
    C1.display();  
    cout<<"C2=";  
    C2.display();  
    cout<<"C3=";  
    C3.display();  
    return 0;  
}
```